

Shopper Digital Twin

Predicting what every customer would buy, before the change happens

A portfolio machine learning project on a simulated general store

Author: Manuga | Version 1.0 | 25 September 2026 | Language: Python

1. Overview

The Shopper Digital Twin builds a simulated copy (a "twin") of every customer in a store from their past browsing and purchases. Each twin knows that shopper's tastes, habits, budget and sensitivity to price. When the business wants to change something, such as raising a price, launching a product or removing one, the change is run on the twins first. The result is a forecast of what real customers would most likely do: who buys more, who switches to a competitor product, and who stops buying altogether.

The project runs on a large, realistic, generated dataset for a general store in the style of Walmart: groceries, household goods, health and beauty, electronics, clothing, toys, home and garden. Because the data is generated, the true behaviour of every shopper is known, which makes it possible to prove how accurate the twins are.

Core concept: prediction. The project answers "what will this person buy next?" and, more ambitiously, "what would this person buy if the world were different?"

2. The problem

Retailers make pricing, range and promotion decisions every week, and each one is expensive to get wrong:

- **Price changes are guesses.** Raise a price too far and customers leave; cut it too far and margin disappears. Most stores use one average price sensitivity for everyone, although real customers differ hugely.
- **Real experiments are slow and risky.** A/B testing a price takes weeks, annoys customers, and only tests one idea at a time.
- **What-if questions can't be checked.** Once you raise a price you never see what would have happened if you hadn't. Forecasts of alternative decisions are rarely validated.
- **Recommenders only look backwards.** Standard recommendation systems predict the next click under today's conditions; they can't say how behaviour shifts when prices, stock or products change.

3. The solution

Give every shopper a personal, learned decision model and a simulator to run it forward in time. The twins act as a safe "virtual market" for testing decisions before making them for real.

Capability	What it delivers
Next-purchase prediction	The products each shopper is most likely to buy next (a recommender).
What-if simulation	Demand, revenue and switching patterns under a price change, new product, stock-out or promotion.
Segment insight	Which kinds of shoppers react most (e.g. budget families vs. premium singles).
Explainable twins	Ask any twin, in plain English, why it did or didn't buy something.
Proven accuracy	Every prediction is scored against the hidden truth of the generated world.

4. Why a general store

A Walmart-style store is the best setting because it shows **both** kinds of behaviour the twins must learn:

Behaviour	Where it shows up	What the twin learns
Repeat buying	Groceries, household, health and beauty, pet food	Repurchase cycles (milk weekly, detergent monthly), brand loyalty, basket habits
Taste	Clothing, electronics, toys, home decor, books	Style and quality preferences, brand affinity, willingness to pay for premium
Price sensitivity	Everywhere, especially store brand vs. name brand	How far each shopper will trade down or leave when prices rise
Cross-category links	Baby products + formula, BBQ + charcoal	Life events and missions that drive whole baskets

5. How it works

The system is a pipeline of seven parts:

#	Part	What it does
1	Synthetic world	Generates ~5,000 products, ~100,000 shoppers and months of sessions. Each shopper has hidden true tastes, price sensitivity and habits, stored as a sealed answer key.
2	Features	Turns raw event logs into product embeddings, shopper histories, prices seen and context (time, season, promotions).
3	Twin model	Learns each shopper's parameters from their behaviour alone. It never sees the answer key.
4	Simulator	Runs every twin day by day: whether they visit, what they browse, what they buy.
5	What-if engine	Applies a change to the world, re-simulates, and compares against the baseline.
6	Evaluation	Scores predictions against held-out behaviour and against the answer key.
7	Demo app	Interactive price slider, segment charts and a chat to interview any twin.

5.1 The twin's decision rule

Each twin uses a **discrete choice model** (multinomial logit), the standard economic model of how people choose between options. For every product it sees, a twin computes a utility score:

$$\text{utility} = \text{taste match} + \text{habit} + \text{context} - \text{price sensitivity} \times \text{price}$$

- **Taste match:** how close the shopper's learned taste vector is to the product's embedding.
- **Habit:** a bonus for favourite brands and for items that are due to be repurchased.
- **Context:** season, day of week, and whether the item is on promotion.
- **Price sensitivity:** a personal value per shopper. This is what makes price what-ifs possible.

The twin turns utilities into purchase probabilities with a softmax, which includes a "**buy nothing**" option. That means the model predicts lost sales as well as switching between products.

5.2 Learning the twins

- **Product embeddings** are learned from what gets browsed and bought together.

- **Shopper parameters** are fitted to explain each shopper's actual choices (maximum likelihood).
- **Hierarchical pooling:** shoppers with little history borrow strength from similar shoppers, relying more on their own data as it grows.
- **Sequence model (upgrade):** a small transformer reads recent clicks to capture what the shopper is doing right now.
- **Uncertainty:** each twin keeps a range for its parameters, so every forecast comes with confidence bands.

5.3 Running the simulation

For each simulated day, all twins are processed at once using vectorised maths:

- **Arrive:** each twin visits with a probability based on its shopping frequency and the season.
- **Browse:** the twin is shown its ~50 most relevant products, found with fast nearest-neighbour search (FAISS).
- **Choose:** utilities are computed, converted to probabilities, and a choice is sampled.
- **Update:** purchases reset repurchase clocks, spend budget, and let tastes drift slightly.

5.4 Asking what-if questions

A scenario is a change to what the twins see: a new price, a new product inserted with its own embedding, a product removed, or a promotion switched on. Baseline and scenario runs use the **same random seeds**, so every difference is caused by the change and not by chance. Outputs include demand change, revenue change, where lost demand went, and which segments reacted.

5.5 The AI-agent layer

A large language model gives each twin a persona built from its learned parameters, for example: "price-sensitive parent, loyal to the store brand for staples, buys premium coffee". Users can ask a twin why it made a choice, and the answer is grounded in that twin's real utility breakdown. The LLM explains; it never changes the numbers, so predictions stay reproducible and cheap to run at scale.

6. How success is measured

The headline test: because the world is generated, the true rules can be re-run with a change applied. That gives the real answer to a what-if question, which retailers can never observe, and the twins' forecast can be scored against it.

Test	Question	Metric
Next purchase	Does the twin know what the shopper buys next?	Hit rate@10, NDCG@10 on held-out weeks
Parameter recovery	Did the model learn the hidden tastes and price sensitivities?	Correlation of learned vs. true values
What-if accuracy	Does the forecast match the true outcome of a change?	Error in predicted vs. actual demand change
Calibration	When the twin says 30%, does it happen 30% of the time?	Calibration curve, Brier score
Baselines	Is it better than simpler methods?	vs. popularity, collaborative filtering, one elasticity for all
Efficiency	Does it run at large-company scale?	Twins simulated per second, retrieval latency

7. Technology

Area	Tools
Language	Python
Data	NumPy, pandas / Polars, Parquet files
Modelling	PyTorch (choice model, embeddings, sequence transformer), scikit-learn (baselines)
Retrieval	FAISS nearest-neighbour search
AI agents	Claude API for twin personas and interviews
Demo	Streamlit app with interactive charts
Engineering	GitHub, pytest, reproducible configs and seeds

8. Roadmap

Milestone	Outcome
1. Synthetic world	General-store catalogue, shoppers, sessions and the answer key, with sanity charts.
2. Baselines	Popularity and matrix-factorisation recommenders to beat.
3. Twin model v1	Per-shopper choice model with price sensitivity and pooling; next-purchase metrics.
4. Simulator + what-if engine	Price, stock-out, new-product and promotion scenarios with confidence bands.
5. Evaluation report	Parameter recovery and what-if accuracy against the answer key.

Milestone	Outcome
6. Upgrades	Sequence transformer, taste drift, life events.
7. Demo app	Price slider, segment breakdown, interview-a-twin chat.
8. Write-up	README and results charts, ready for the portfolio.

9. Scale targets and limits

- 100,000 shoppers, 5,000 products and around 10 million events, generated in minutes.
- Training in under an hour on a laptop; a 30-day what-if for all twins in seconds.
- **Limit:** results are proven on synthetic data. The generator is designed to be realistic, but real shoppers are messier. The same pipeline would work on a real retailer's logs.
- **Limit:** the model predicts how shoppers react to the store's own changes; competitor moves are outside scope for v1.